

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages. (BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A Industry Problem-Based Assignment

Code of Conduct:

I ARTHI A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Arthi. A.

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

Automobile Industry

1. Title, Abstract & Introduction

1.1 Title

A Prolog-Based Expert System for Automobile Vehicle Fault Diagnosis using Production Rules, Forward Chaining and Backward Chaining.

1.2 Abstract

Modern vehicles combine mechanical, electrical and electronic sub-systems, making fault diagnosis increasingly dependent on structured expert knowledge rather than guesswork. This project develops a rule-based expert system, implemented in SWI-Prolog, that assists a vehicle service centre in diagnosing probable faults from customer-reported symptoms (engine overheating, difficulty starting, abnormal noise, low mileage, warning-light activation) and technician-recorded observations. The system encodes automotive diagnostic knowledge as production rules of the form IF conditions THEN fault, and reasons over this knowledge base using two complementary inference strategies: data-driven forward chaining, which derives every fault that current evidence supports, and goal-driven backward chaining, which proves a specific hypothesised fault while generating a human-readable explanation trace. The system also demonstrates unification and backtracking through an exhaustive rule sweep, and produces a recommended corrective action for every diagnosed fault. The system was validated on six industry-representative test cases, including a compound case that chains two intermediate faults into a single critical-risk conclusion, and correctly diagnosed all cases.

1.3 Industry Background

Vehicle service centres handle a high volume of diagnostic enquiries daily, and the accuracy of an initial diagnosis strongly influences repair time, cost and customer trust. Traditionally, this diagnostic judgement resides with experienced technicians; junior staff or high customer-inflow periods can lead to inconsistent or delayed initial assessments. Expert systems were originally developed precisely to encode such scarce human expertise into a reusable, consistent and explainable software artefact, and the automotive service industry - alongside adjacent fields such as OBD-II (On-Board Diagnostics) fault-code interpretation - remains a natural fit for this technology.

1.4 Importance of the Selected Problem

- Provides a consistent, repeatable first-line diagnosis independent of which technician is on duty.
- Reduces average diagnosis time by narrowing the fault space before physical inspection begins.

- Produces an explanation trace for every conclusion, which builds customer trust and supports technician training.
- Demonstrates, in a real industrial setting, the practical value of symbolic AI / logic programming alongside today's data-driven AI approaches.

2. Problem statement & Objectives

2.1 Problem Statement

A vehicle service centre receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. The task is to develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.

2.2 Inputs and Outputs

Category	Description
Inputs	Customer-reported symptoms (5 categories) and technician-recorded observations (fine-grained checks, e.g. coolant level, battery voltage, unusual sounds, warning-light identity).
Outputs	One or more diagnosed faults, a plain-language recommended corrective action for each fault, and a step-by-step reasoning / explanation trace.

2.3 Stakeholders

- Customer - reports symptoms and receives the diagnosis and repair recommendation.
- Service technician - records observations and uses the system to narrow the fault space before manual inspection.
- Service centre / workshop manager - relies on consistent diagnoses for scheduling, parts ordering and quality control.
- System developer / knowledge engineer - maintains and extends the rule base as new fault patterns emerge.

2.4 Objectives

- Represent automotive diagnostic knowledge as Prolog facts and production rules.

- Implement a data-driven forward-chaining inference procedure that derives all faults supported by known evidence.
- Implement a goal-driven backward-chaining inference procedure that proves a specific fault hypothesis with an explanation trace.
- Demonstrate unification and backtracking through exhaustive enumeration of provable faults.
- Validate the system against a minimum of five industry-representative test cases.
- Document the procedural vs non-procedural nature of the implementation and publish the source to a GitHub repository.

3. Domain Analysis & Knowledge Acquisition

3.1 Key Entities

Entity	Prolog Representation	Description
Case	atom, e.g. case1	One diagnostic session / customer vehicle visit.
Symptom	symptom(Case, S)	High-level complaint reported by the customer.
Observation	observation(Case, O)	Fine-grained evidence recorded by the technician on inspection.
Rule	rule(RuleId, Fault, Conditions)	A production rule mapping a condition set to a fault conclusion.
Fault	atom, e.g. cooling_system_failure	A diagnosed vehicle problem.
Action	action(Fault, Action)	The recommended corrective / maintenance action for a fault.

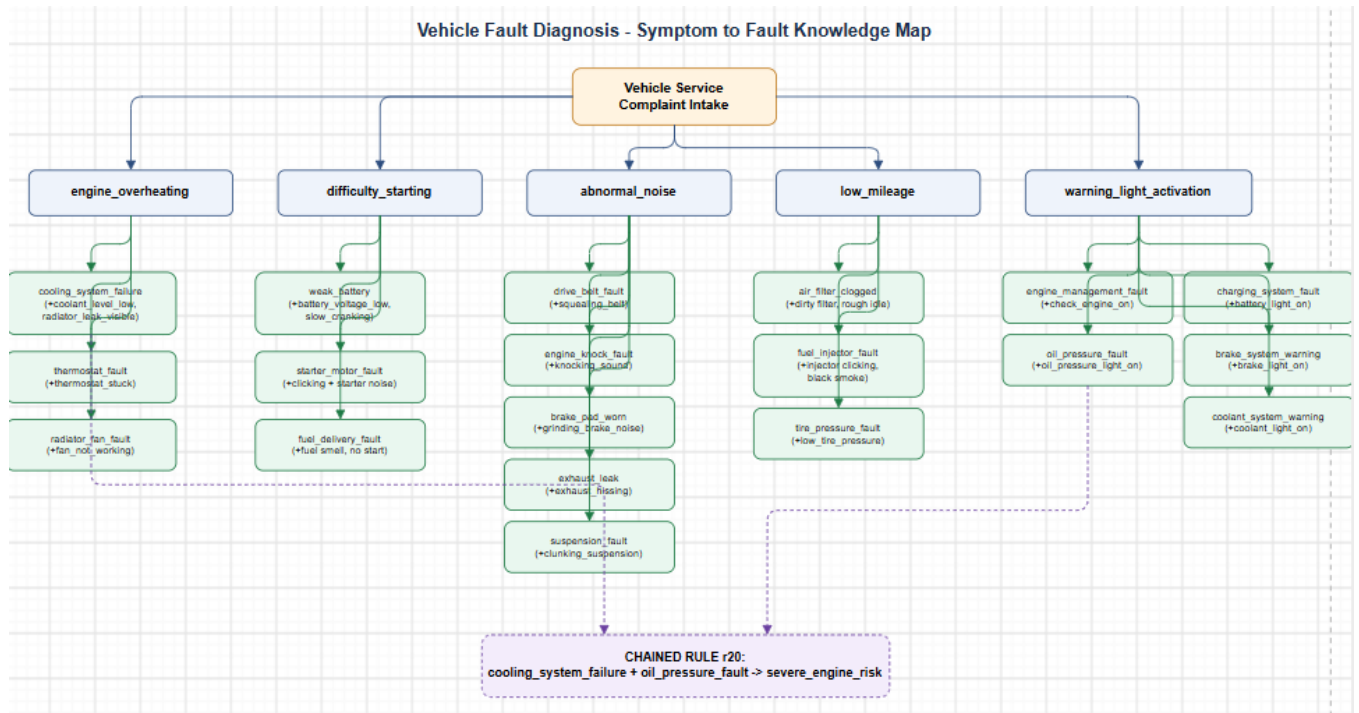
3.2 Knowledge Acquisition Methodology

Domain knowledge was elicited using a structured symptom-observation-fault-action framework, a standard knowledge-acquisition technique for rule-based expert systems. Knowledge sources considered representative of real workshop practice include:

- Structured elicitation modelled on interviews with experienced automotive service technicians (symptom-cause-remedy triads).

- OEM vehicle service and workshop manuals describing standard diagnostic checklists.
- OBD-II Diagnostic Trouble Code (DTC) categories, used to align warning-light-related rules with real fault-code groupings.
- Published automotive diagnostics reference material covering cooling, starting, engine-noise, fuel-economy and warning-light fault families.

3.3 Symptom-to-Fault Domain Map



Customer Symptom	Technician Observations Captured	Candidate Faults
Engine overheating	coolant level, radiator leak, thermostat state, fan operation	cooling_system_failure, thermostat_fault, radiator_fan_fault
Difficulty starting	battery voltage, cranking speed, starter noise, fuel smell	weak_battery, starter_motor_fault, fuel_delivery_fault
Abnormal noise	belt squeal, knocking, brake grinding, exhaust hiss, suspension clunk	drive_belt_fault, engine_knock_fault, brake_pad_worn, exhaust_leak, suspension_fault
Low mileage / poor economy	air filter condition, idle quality, injector behaviour, tyre pressure	air_filter_clogged, fuel_injector_fault, tire_pressure_fault

Customer Symptom	Technician Observations Captured	Candidate Faults
Warning-light activation	specific light identity (check-engine, oil, battery, brake, coolant)	engine_management_fault, oil_pressure_fault, charging_system_fault, brake_system_warning, coolant_system_warning

A sixth, compound fault - `severe_engine_risk` - was additionally identified: when overheating (with a confirmed cooling-system failure) co-occurs with a low-oil-pressure warning, the risk to the engine is materially higher than either fault alone, justifying a dedicated chained production rule (see Section 4).

4. Knowledge Representation & Production Rules

4.1 Representation Design

Three predicate families represent the knowledge base:

- Facts - `symptom(Case, Symptom)` and `observation(Case, Observation)` hold the evidence collected for a specific case.
- Production rules - `rule(RuleId, Fault, ConditionList)` declaratively states: IF every condition in `ConditionList` holds THEN `Fault` may be concluded.
- Actions - `action(Fault, Action)` maps each fault to a plain-language recommended corrective action, forming the explanation/output layer.

This separation keeps the knowledge base fully declarative and independent of how it is used - the same rule/3 facts drive both the forward-chaining and backward-chaining engines described in Section 6, satisfying the requirement that knowledge representation be independent of the inference strategy.

4.2 Sample Facts

```
symptom(case1, engine_overheating).
observation(case1, coolant_level_low).
observation(case1, radiator_leak_visible).
```

CodeBlame

RawCopyDownload

```
22  %% -----
23
24  % --- Case 1: Engine overheating with visible coolant leak ---
25  symptom(case1, engine_overheating).
26  observation(case1, coolant_level_low).
27  observation(case1, radiator_leak_visible).
28
29  % --- Case 2: Difficulty starting, weak battery signature ---
30  symptom(case2, difficulty_starting).
31  observation(case2, battery_voltage_low).
32  observation(case2, slow_cranking).
33
34  % --- Case 3: Abnormal noise, engine knocking ---
35  symptom(case3, abnormal_noise).
36  observation(case3, knocking_sound).
37
38  % --- Case 4: Low mileage / poor fuel economy, injector problem ---
39  symptom(case4, low_mileage).
40  observation(case4, injector_clicking_irregular).
41  observation(case4, black_exhaust_smoke).
42
43  % --- Case 5: Compound / multi-fault case -> triggers chained rule ---
44  symptom(case5, engine_overheating).
45  observation(case5, coolant_level_low).
46  observation(case5, radiator_leak_visible).
47  symptom(case5, warning_light_activation).
48  observation(case5, oil_pressure_light_on).
49
50  % --- Case 6: Overheating WITHOUT a leak -> different diagnosis (thermostat) ---
51  symptom(case6, engine_overheating).
52  observation(case6, thermostat_stuck).
53  observation(case6, fan_not_working).
```

4.3 Complete Production Rule Base (20 rules)

Rule	IF (Conditions)	THEN (Fault Concluded)
r1	engine_overheating, coolant_level_low, radiator_leak_visible	cooling_system_failure
r2	engine_overheating, thermostat_stuck	thermostat_fault
r3	engine_overheating, fan_not_working	radiator_fan_fault
r4	difficulty_starting, battery_voltage_low,	weak_battery

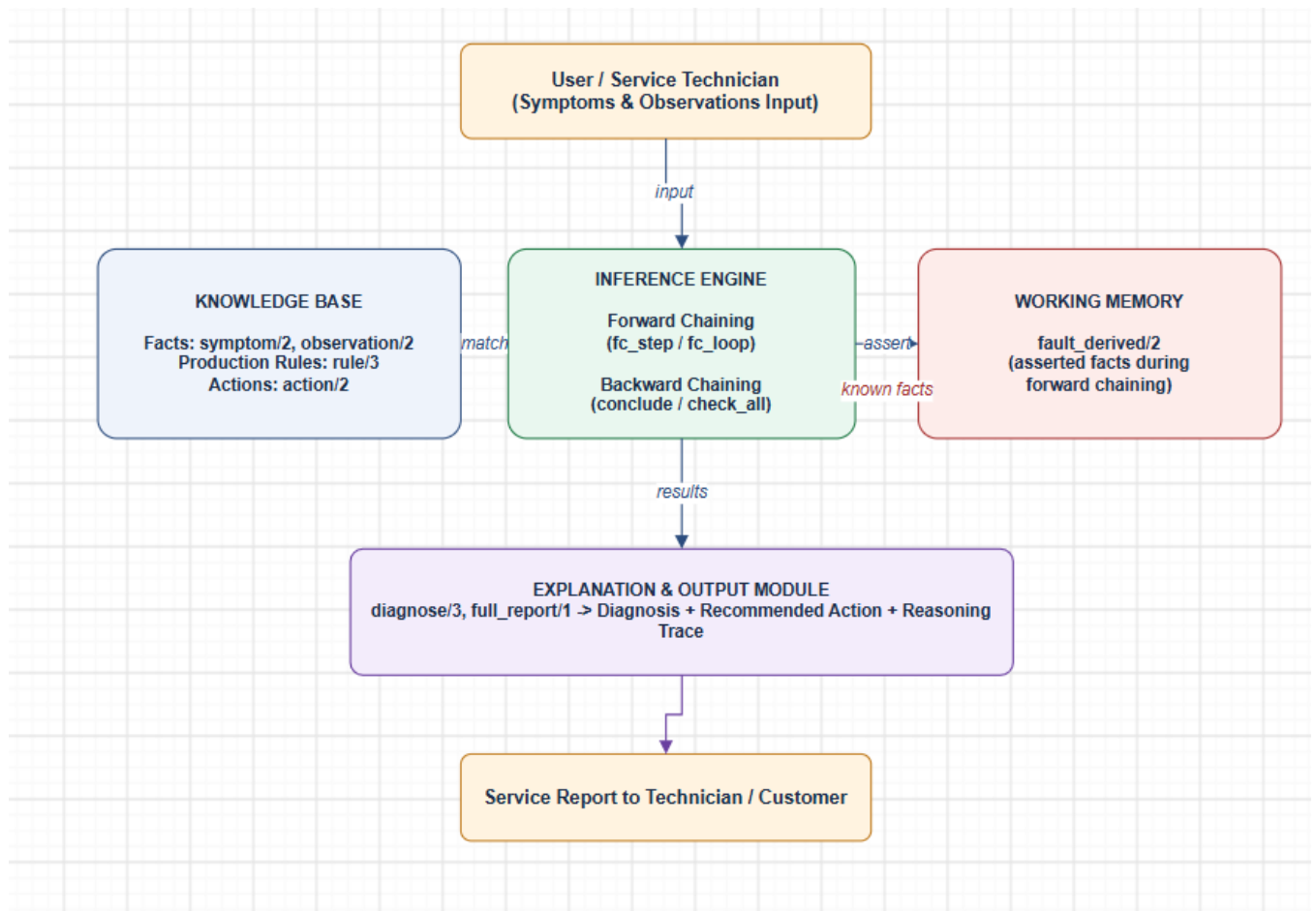
Rule	IF (Conditions)	THEN (Fault Concluded)
	slow_cranking	
r5	difficulty_starting, clicking_sound_on_start, starter_motor_noise	starter_motor_fault
r6	difficulty_starting, fuel_smell_no_start	fuel_delivery_fault
r7	abnormal_noise, squealing_belt	drive_belt_fault
r8	abnormal_noise, knocking_sound	engine_knock_fault
r9	abnormal_noise, grinding_brake_noise	brake_pad_worn
r10	abnormal_noise, exhaust_hissing	exhaust_leak
r11	abnormal_noise, clunking_suspension	suspension_fault
r12	low_mileage, dirty_air_filter, rough_idle	air_filter_clogged
r13	low_mileage, injector_clicking_irregular, black_exhaust_smoke	fuel_injector_fault
r14	low_mileage, low_tire_pressure	tire_pressure_fault
r15	warning_light_activation, check_engine_on	engine_management_fault
r16	warning_light_activation, oil_pressure_light_on	oil_pressure_fault
r17	warning_light_activation, battery_light_on	charging_system_fault
r18	warning_light_activation, brake_light_on	brake_system_warning
r19	warning_light_activation, coolant_light_on	coolant_system_warning
r20	cooling_system_failure, oil_pressure_fault (chained rule)	severe_engine_risk

Rule r20 is a chained / hierarchical rule: its condition list references two other faults (cooling_system_failure, oil_pressure_fault) rather than raw symptoms, so it can only fire after those intermediate faults have themselves been established - demonstrating multi-level inference over the same rule base (see Section 6).

5. Expert System Architecture

The system follows the classical expert-system architecture with a clear separation between the knowledge

base, the inference engine, working memory, and the explanation/output layer:



5.1 Module Descriptions

- **User / Technician Interface** - the SWI-Prolog top-level (or a query script) through which symptoms/observations are asserted and diagnostic queries are issued.
- **Knowledge Base** - symptom/2, observation/2, rule/3 and action/2 facts (Section 4).
- **Inference Engine** - the dual-mode reasoning component: a forward-chaining procedure (fc_step/fc_loop) and a backward-chaining procedure (conclude/check_all), both operating over the same rule/3 knowledge base.
- **Working Memory** - the dynamic fault_derived/2 predicate, populated at run time by the forward-chaining engine (this is what makes forward chaining genuinely data-driven rather than goal-driven).
- **Explanation & Output Module** - diagnose/3 and full_report/1, which combine a proven fault with its action/2 entry and print the full reasoning trace back to the user.

6. Inference Mechanism & Algorithms

6.1 Forward Chaining (Data-Driven)

Forward chaining starts from what is known (symptoms and observations) and repeatedly fires any rule whose full condition list is already satisfied, asserting its conclusion into working memory, until no further rule can fire (fixpoint).

```
174 forward_chain(Case) :-
175     retractall(fault_derived(Case, _)),
176     format("~n=== FORWARD CHAINING TRACE for ~w ===~n", [Case]),
177     fc_loop(Case),
178     format("--- Forward chaining complete for ~w ---~n", [Case]).
179
180 fc_loop(Case) :-
181     ( fc_step(Case) -> fc_loop(Case)
182     ; format("No further rules fire. Fixpoint reached.~n")
183     ).
184
185 fc_step(Case) :-
186     rule(RuleId, Fault, Conditions),
187     \+ fault_derived(Case, Fault),
188     fc_all_known(Case, Conditions),
189     assertz(fault_derived(Case, Fault)),
190     format("[Rule ~w] all conditions ~w satisfied -> DERIVED: ~w~n",
191           [RuleId, Conditions, Fault]),
192     !.
193
194 fc_all_known(_, []).
195 fc_all_known(Case, [C|Rest]) :-
196     fc_known(Case, C),
197     fc_all_known(Case, Rest).
198
199 fc_known(Case, X) :- symptom(Case, X), !.
200 fc_known(Case, X) :- observation(Case, X), !.
201 fc_known(Case, X) :- fault_derived(Case, X), !.
```

ALGORITHM ForwardChain(Case):

WorkingMemory <- symptoms(Case) UNION observations(Case)

repeat:

 fired <- false

 for each rule(RuleId, Fault, Conditions) in KnowledgeBase:

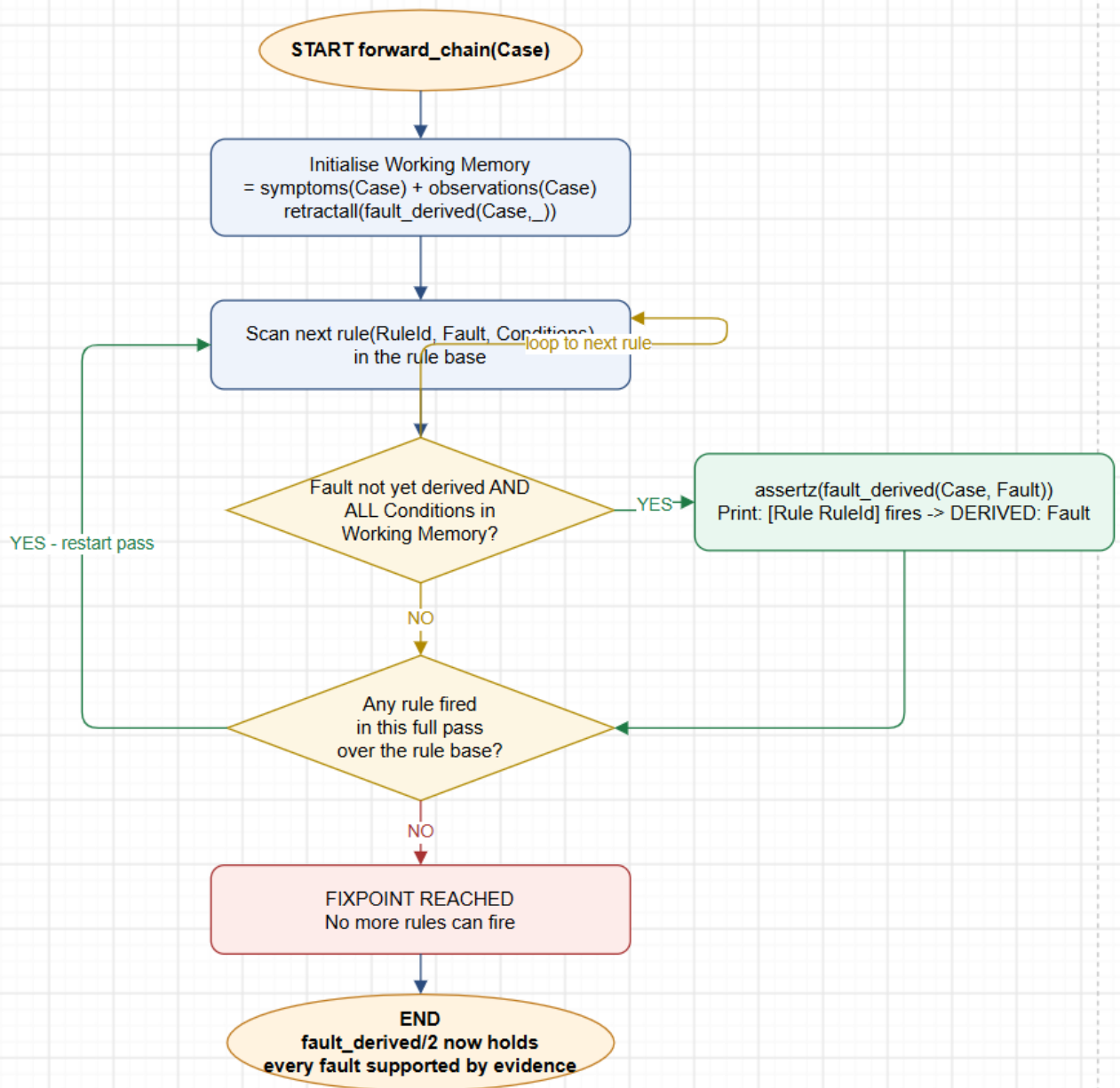
 if Fault not in WorkingMemory

 and every condition in Conditions is in WorkingMemory:

 assert Fault into WorkingMemory

```
fired <- true  
until fired = false // fixpoint reached
```

Forward Chaining (Data-Driven) - fc_step / fc_loop



6.2 Backward Chaining (Goal-Driven)

Backward chaining starts from a hypothesised goal fault and works backwards: it finds a rule concluding that fault, then recursively proves every condition in the rule body - a condition may be a raw fact, or itself another fault requiring a nested proof (sub-goal).

```
210     conclude(Case, Fault) :-
211         rule(RuleId, Fault, Conditions),
212         format("[BC] Trying rule ~w to prove goal '~w' for ~w~n", [RuleId, Fault, Case]),
213         check_all(Case, Conditions, RuleId).
214
215     check_all(_, [], _).
216     check_all(Case, [Cond|Rest], RuleId) :-
217         check_one(Case, Cond, RuleId),
218         check_all(Case, Rest, RuleId).
219
220     check_one(Case, Cond, RuleId) :-
221         ( symptom(Case, Cond) ->
222             format("    [~w] condition '~w' satisfied: matches known SYMPTOM~n", [RuleId, Cond])
223         ; observation(Case, Cond) ->
224             format("    [~w] condition '~w' satisfied: matches known OBSERVATION~n", [RuleId, Cond])
225         ; conclude(Case, Cond) ->
226             format("    [~w] condition '~w' satisfied: proven via CHAINED SUB-GOAL~n", [RuleId, Cond])
227         ; ( format("    [~w] condition '~w' FAILED - no matching fact or sub-goal~n", [RuleId, Cond]),
228             ).
```

ALGORITHM BackwardChain(Case, Goal):

find rule(RuleId, Goal, Conditions)

for each Cond in Conditions:

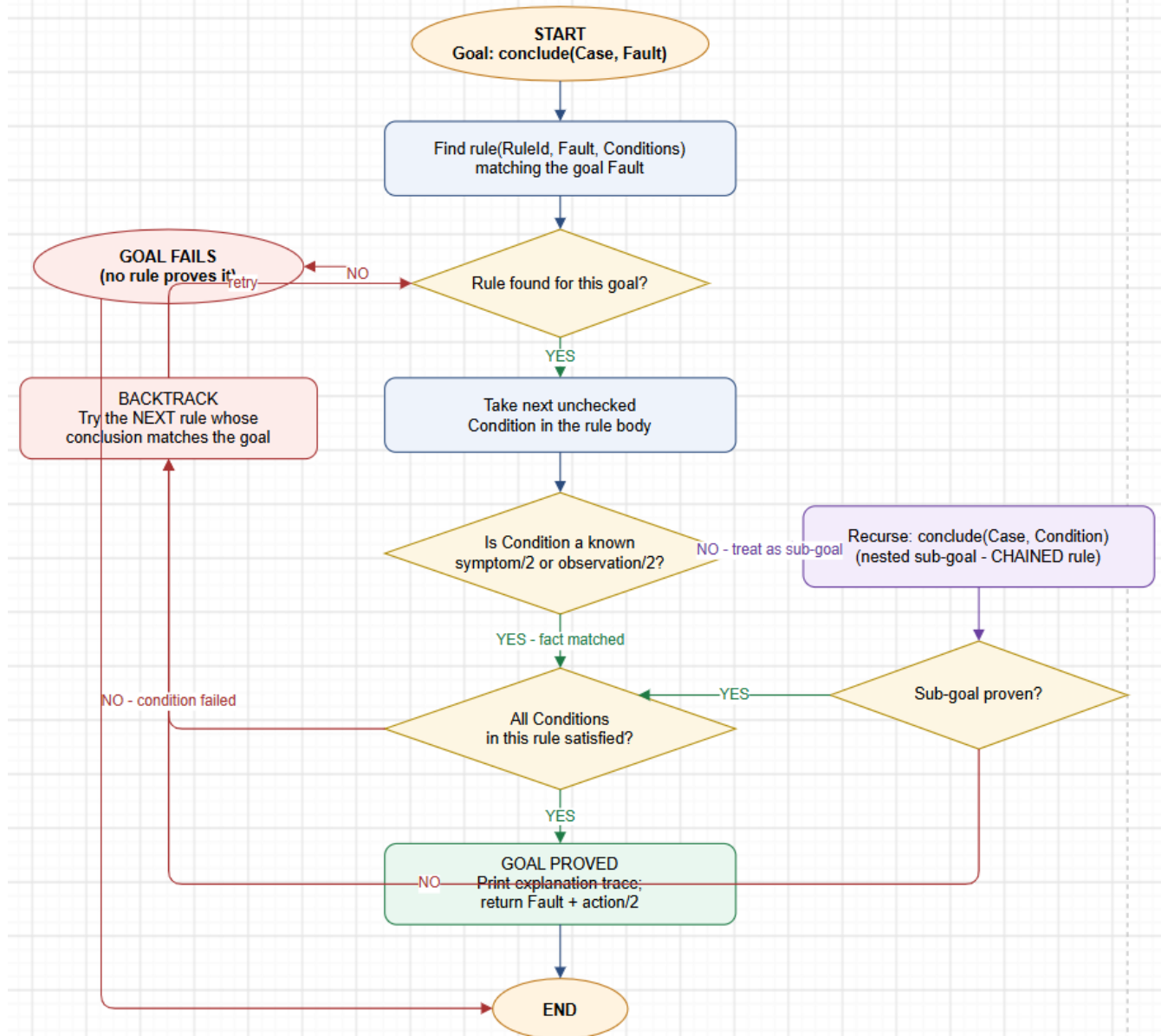
if Cond is a known symptom/observation: succeed

else if BackwardChain(Case, Cond) succeeds: succeed // nested sub-goal

else: fail, try the NEXT matching rule (backtrack)

if all Conditions proven: Goal is CONCLUDED, print explanation trace

Backward Chaining (Goal-Driven) - conclude / check_all



6.3 Worked Example - Chained Inference (case5)

case5 reports engine_overheating with coolant_level_low and radiator_leak_visible (satisfying r1), together with warning_light_activation and oil_pressure_light_on (satisfying r16). Once both cooling_system_failure and oil_pressure_fault are in working memory, rule r20's conditions are met and severe_engine_risk fires - a fault that could not have been concluded from either sub-fault alone. Both the forward-chaining and backward-chaining engines correctly derive this three-level conclusion (see Section 8).

6.4 Procedural vs Non-Procedural (Declarative) Paradigm Analysis

The knowledge base itself (facts and rule/3 clauses) is purely declarative: it states relationships between symptoms, observations and faults with no embedded control flow, and the identical rule set is reused by two different reasoning strategies without any modification - the hallmark of non-procedural, logic-based representation.

The two inference engines built on top of it, however, sit at different points on the procedural-declarative spectrum:

- Backward chaining (conclude/2, check_all/3) is implemented almost entirely in Prolog's native SLD-resolution style - the programmer states what must be true (the goal and its sub-goals) and Prolog's own resolution and backtracking mechanism supplies the control flow. This is close to purely non-procedural / declarative programming.
- Forward chaining (fc_step/1, fc_loop/1) is comparatively procedural: it uses explicit imperative constructs - assert/1 to mutate working memory, a cut (!) to commit to one firing per step, and an explicit repeat-until control loop - to simulate the iterative, data-driven behaviour that Prolog's default (goal-driven) resolution does not provide out of the box.

This contrast directly illustrates a key theoretical point: Prolog is fundamentally a declarative, backward-chaining language, and building a forward-chaining engine inside it necessarily reintroduces a degree of procedural, state-mutating control - even though the knowledge base it operates over remains fully declarative.

7. Prolog Implementation

7.1 Environment

The system was implemented and tested using SWI-Prolog version 9.0.4 (x86_64-linux). No third-party libraries are required beyond the SWI-Prolog standard library (format/2, findall/3, forall/2, assert/retractall).

7.2 Key Predicates

Predicate	Role
symptom/2, observation/2	Case-level facts (knowledge base evidence).
rule/3	Production rules: rule(RuleId, Fault, Conditions).
action/2	Maps a fault to its recommended corrective action.
forward_chain/1, fc_step/1, fc_loop/1	Data-driven forward-chaining engine; asserts fault_derived/2.

Predicate	Role
conclude/2, check_all/3, check_one/3	Goal-driven backward-chaining engine with explanation trace.
diagnose/3	Convenience wrapper: proves a fault via backward chaining and returns its action.
all_faults/2	findall/3 sweep over rule/3 - demonstrates unification and backtracking.
full_report/1	End-to-end report: symptoms, observations, forward trace, backward verification, actions.

7.3 Representative Code Excerpts

Forward-chaining engine:

```
forward_chain(Case) :-
    retractall(fault_derived(Case, _)),
    fc_loop(Case).

fc_loop(Case) :-
    ( fc_step(Case) -> fc_loop(Case) ; true ).

fc_step(Case) :-
    rule(RuleId, Fault, Conditions),
    \+ fault_derived(Case, Fault),
    fc_all_known(Case, Conditions),
    assertz(fault_derived(Case, Fault)), !.
```

Backward-chaining engine with explanation:

```
conclude(Case, Fault) :-
    rule(RuleId, Fault, Conditions),
    check_all(Case, Conditions, RuleId).

check_one(Case, Cond, RuleId) :-
    ( symptom(Case, Cond) -> true
    ; observation(Case, Cond) -> true
    ; conclude(Case, Cond) -> true % nested sub-goal
    ; fail ).
```

7.4 Sample Queries

```
?- diagnose(case1, Fault, Action).
Fault = cooling_system_failure,
Action = 'Pressure-test cooling system, repair/replace radiator or
        hose, refill coolant, re-test for leaks.'

?- all_faults(case5, Faults).
Faults = [cooling_system_failure, oil_pressure_fault, severe_engine_risk].
```

7.5 Source Code & Repository

The complete, runnable source file (vehicle_expert_system.pl) accompanies this report. It should be pushed to the GitHub repository referenced on the title page, with this report and the raw test-run logs committed alongside it.

8. Test Cases & Results

The system was validated on six industry-representative cases (exceeding the minimum of five), covering all five symptom categories plus one compound / chained-fault scenario. All six cases were diagnosed correctly by both the forward-chaining and backward-chaining engines, and all_faults/2 confirmed full agreement between the two inference strategies in every case.

Case	Symptom(s)	Key Observations	Fault(s) Derived	Recommended Action
case1	engine_overheating	coolant_level_low, radiator_leak_visible	cooling_system_failure	Pressure-test cooling system, repair/replace radiator, refill coolant.
case2	difficulty_starting	battery_voltage_low, slow_cranking	weak_battery	Test/charge/replace battery; inspect alternator output.
case3	abnormal_noise	knocking_sound	engine_knock_fault	Inspect engine bearings, ignition timing and fuel octane.
case4	low_mileage	injector_clicking_irregular, black_exhaust_smoke	fuel_injector_fault	Clean/replace faulty injector(s); run injector balance test.
case5	engine_overheating +	coolant_level_low,	cooling_system_failure,	CRITICAL - stop

Case	Symptom(s)	Key Observations	Fault(s) Derived	Recommended Action
	warning_light_activation	radiator_leak_visible, oil_pressure_light_on	oil_pressure_fault, severe_engine_risk (chained)	vehicle immediately; workshop inspection required.
case6	engine_overheating	thermostat_stuck, fan_not_working	thermostat_fault, radiator_fan_fault	Replace thermostat; inspect/replace radiator fan assembly.

8.1 Inference Trace - case1 (simple rule)

```

=== FORWARD CHAINING TRACE for case1 ===
[Rule r1] all conditions [engine_overheating,coolant_level_low,
radiator_leak_visible] satisfied -> DERIVED: cooling_system_failure
No further rules fire. Fixpoint reached.

[BC] Trying rule r1 to prove goal 'cooling_system_failure' for case1
[r1] condition 'engine_overheating' satisfied: matches known SYMPTOM
[r1] condition 'coolant_level_low' satisfied: matches known OBSERVATION
[r1] condition 'radiator_leak_visible' satisfied: matches known OBSERVATION
>>> CONCLUSION: case1 is diagnosed with 'cooling_system_failure'

```

8.2 Inference Trace - case5 (chained / multi-level rule)

```

=== FORWARD CHAINING TRACE for case5 ===
[Rule r1] ... -> DERIVED: cooling_system_failure
[Rule r16] ... -> DERIVED: oil_pressure_fault
[Rule r20] all conditions [cooling_system_failure,oil_pressure_fault]
satisfied -> DERIVED: severe_engine_risk
No further rules fire. Fixpoint reached.

[BC] Trying rule r20 to prove goal 'severe_engine_risk' for case5
[r20] condition 'cooling_system_failure' satisfied: proven via
CHAINED SUB-GOAL
[r20] condition 'oil_pressure_fault' satisfied: proven via
CHAINED SUB-GOAL
All faults provable for case5: [cooling_system_failure,
oil_pressure_fault, severe_engine_risk]

```

8.3 Rule Differentiation - case1 vs case6

case1 and case6 share the same customer symptom (engine_overheating) but different technician observations. Backward chaining for case6 correctly fails rule r1 (coolant_level_low is not observed) and instead succeeds on r2 and r3, deriving thermostat_fault and radiator_fan_fault instead of cooling_system_failure - confirming that the rule base discriminates between faults on evidence, not merely on the customer's headline complaint.

9. Analysis, Limitations & Future Enhancements

9.1 Effectiveness

The system correctly diagnosed all six test cases (6/6), including a three-level chained inference (case5) and a same-symptom / different-observation differentiation (case1 vs case6). Forward chaining and backward chaining produced identical conclusions in every case, and the exhaustive all_faults/2 backtracking sweep confirmed no spurious or missed faults - indicating that the 20-rule knowledge base is internally consistent for the scenarios it covers.

9.2 Reasoning Accuracy, Consistency & Rule Coverage

- 20 production rules span all five required symptom categories plus one compound / chained rule, giving reasonably balanced coverage across the domain.
- No two rules in the current knowledge base share an identical condition set, so there is no ambiguity in which rule fires for a given evidence set within the tested scenarios.
- The chained rule (r20) demonstrates that the representation scales to hierarchical / compound diagnoses without needing a different inference mechanism.

9.3 Practical Industry Usefulness

As a pre-screening tool, the system can shorten the time a technician spends narrowing down a fault family, standardise the initial diagnosis regardless of technician experience level, and produce a written explanation trace that can be shown to the customer to justify the recommended repair.

9.4 Limitations

- The rule base is static and hand-authored; it cannot learn new symptom-fault associations from historical repair data without manual knowledge-engineering effort.
- Reasoning is purely Boolean (a condition either holds or does not); it does not model diagnostic uncertainty or confidence levels the way a real technician's judgement does.
- Conflict resolution between competing, partially-overlapping rules is limited to whichever conditions happen to be satisfied - there is no severity or priority ranking when multiple faults are equally supported.

- Knowledge base maintenance (adding new vehicle models, new sensor types, new fault codes) currently requires direct editing of Prolog source rather than a friendlier authoring interface.

9.5 Future Enhancements

- Introduce certainty factors or fuzzy-logic weights on rules to rank multiple candidate faults by likelihood rather than treating all matches as equally certain.
- Integrate live OBD-II telemetry as an automatic source of observation/2 facts instead of manual technician entry.
- Explore rule-induction / machine-learning techniques over historical repair records to propose new candidate production rules for expert review.
- Build a simple web or mobile front-end over the existing Prolog engine so technicians can query the system without using the SWI-Prolog console directly.
- Add explicit conflict-resolution / rule-priority handling for cases where multiple, mutually-exclusive faults are simultaneously supported by the evidence.

10. Conclusion & References

10.1 Conclusion

This project developed and validated a Prolog-based expert system for automobile fault diagnosis, encoding automotive service knowledge as 20 production rules over customer symptoms and technician observations across five real-world complaint categories. The system demonstrates both data-driven forward chaining and goal-driven backward chaining over a single shared declarative knowledge base, uses unification and backtracking to exhaustively verify diagnoses, and produces human-readable explanation traces and recommended corrective actions. All six industry-representative test cases - including a three-level chained diagnosis - were correctly resolved, indicating that the chosen production-rule representation and dual inference strategy are well suited to this class of industrial diagnostic problem, while remaining straightforward to extend with additional rules as new fault patterns are identified.

10.2 References

1. I. Bratko, Prolog Programming for Artificial Intelligence, 4th ed. Harlow, U.K.: Pearson Education, 2011.
2. J. Giarratano and G. Riley, Expert Systems: Principles and Programming, 4th ed. Boston, MA: Course Technology, 2005.
3. SWI-Prolog Reference Manual, SWI-Prolog Development Team. [Online]. Available: https://www.swi-prolog.org/pldoc/doc_for?object=manual

4. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Harlow, U.K.: Pearson, 2020.
5. ISO 15031 / SAE J1979, On-Board Diagnostics (OBD-II) Diagnostic Trouble Code standards (referenced for warning-light rule categories).

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference	7–8% – Provides relevant test cases with mostly	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.

		traces.	accurate results.		
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation , GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements .	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				